Name : Vatsal Postiwala PRN No: 26070122509 Batch : B3(2025-2029)

ASSIGNMENT No: 28

TITLE: Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Problem Statement

Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives

To implement Remote Method Invocation for distributed Java applications.

Theory

Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine. RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication. The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.

Code:

Greeting.java

```
package E28;
import java.lang.*;
import java.rmi.*;

public interface Greeting extends Remote {
   String sayHello() throws RemoteException;
}
```

GreetingImpl.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.server.*;

public class GreetingImpl extends UnicastRemoteObject implements Greeting {
```

```
    public GreetingImpl() throws RemoteException {
        super();
    }
    public String sayHello() {
        return "Hello from the RMI Server!";
    }
}
```

GreetingServer.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.registry.*;

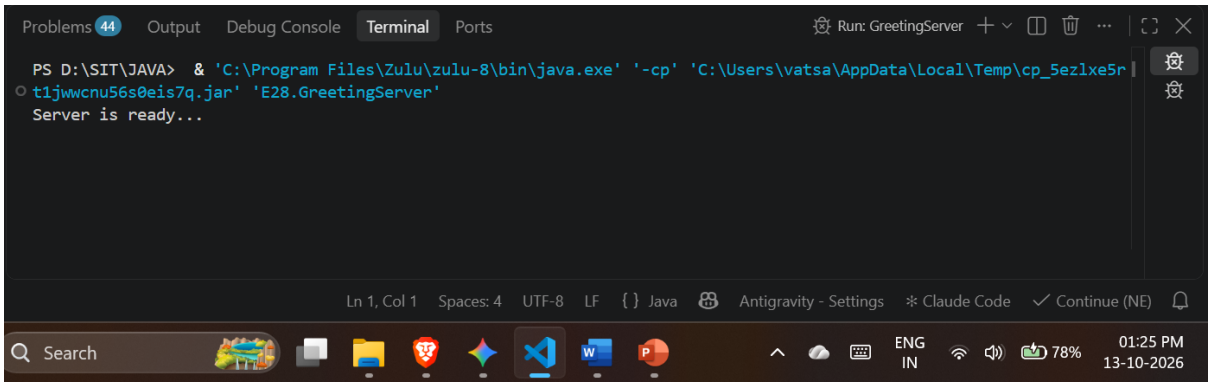
public class GreetingServer {
    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Naming.rebind("GreetingService", new GreetingImpl());
        System.out.println("Server is ready...");
    }
}
```

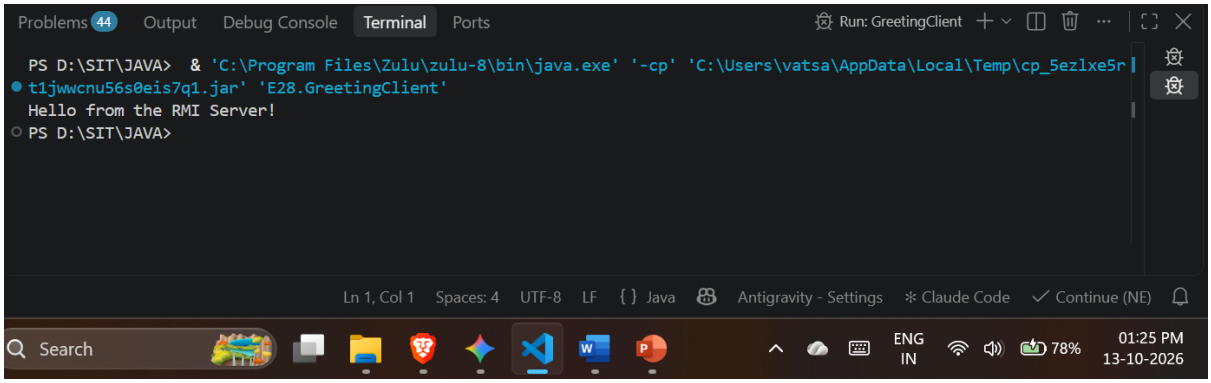
GreetingClient.java

```
package E28;
import java.lang.*;
import java.rmi.*;

public class GreetingClient {
    public static void main(String[] args) throws Exception {
        Greeting g = (Greeting) Naming.lookup("rmi://localhost/GreetingService");
        System.out.println(g.sayHello());
    }
}
```

Output:





Exercise

1. Create a remote calculator service using Java RMI

Code:

Calculator.java

```
package E28;
import java.lang.*;
import java.rmi.*;

public interface Calculator extends Remote {
   int add(int a, int b) throws RemoteException;
   int sub(int a, int b) throws RemoteException;
   int mul(int a, int b) throws RemoteException;
   double div(int a, int b) throws RemoteException;
}
```

CalculatorImpl.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.server.*;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator
{
   public CalculatorImpl() throws RemoteException {
      super();
   }
   public int add(int a, int b) { return a + b; }
   public int sub(int a, int b) { return a - b; }
```

```
    public int mul(int a, int b) { return a * b; }
    public double div(int a, int b) { return (double) a / b; }
}
```

CalcServer.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.registry.*;

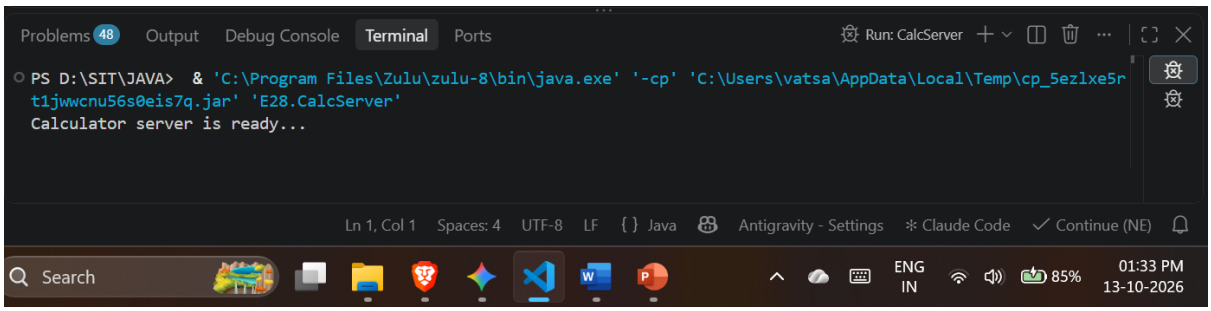
public class CalcServer {
    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Naming.rebind("CalcService", new CalculatorImpl());
        System.out.println("Calculator server is ready...");
    }
}
```

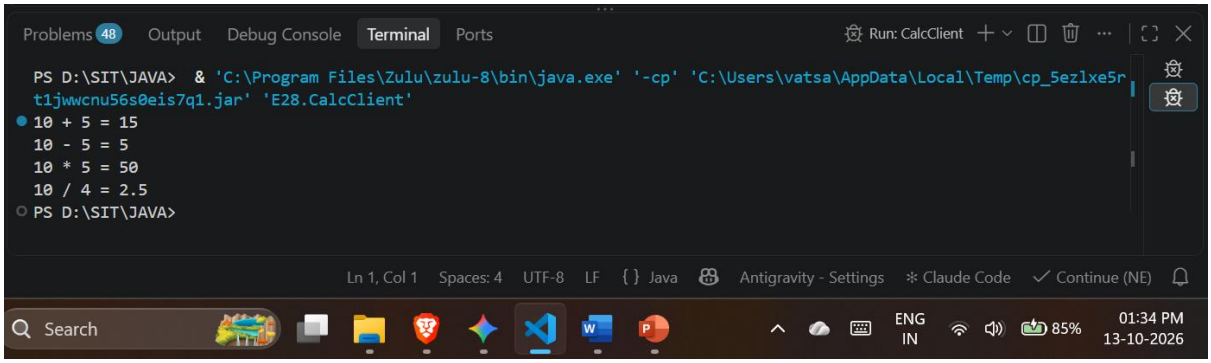
CalcClient.java

```
package E28;
import java.lang.*;
import java.rmi.*;

public class CalcClient {
    public static void main(String[] args) throws Exception {
        Calculator c = (Calculator) Naming.lookup("rmi://localhost/CalcService");
        System.out.println("10 + 5 = " + c.add(10, 5));
        System.out.println("10 - 5 = " + c.sub(10, 5));
        System.out.println("10 * 5 = " + c.mul(10, 5));
        System.out.println("10 / 4 = " + c.div(10, 4));
    }
}
```

Output:





2. Create a Java application using **RMI** where the server converts temperature values between **Celsius and Fahrenheit**. The client sends input temperature and receives converted output.

Code:

Converter.java

```
package E28;
import java.lang.*;
import java.rmi.*;

public interface Converter extends Remote {
   double celsiusToFahrenheit(double c) throws RemoteException;
   double fahrenheitToCelsius(double f) throws RemoteException;
}
```

ConverterImpl.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.server.*;

public class ConverterImpl extends UnicastRemoteObject implements Converter {
   public ConverterImpl() throws RemoteException {
      super();
   }
   public double celsiusToFahrenheit(double c) {
      return (c * 9 / 5) + 32;
   }
   public double fahrenheitToCelsius(double f) {
      return (f - 32) * 5 / 9;
   }
}
```

TempServer.java

```
package E28;
import java.lang.*;
import java.rmi.*;
import java.rmi.registry.*;

public class TempServer {
   public static void main(String[] args) throws Exception {
      LocateRegistry.createRegistry(1099);
      Naming.rebind("TempService", new ConverterImpl());
      System.out.println("Temperature server is ready...");
   }
}
```

TempClient.java

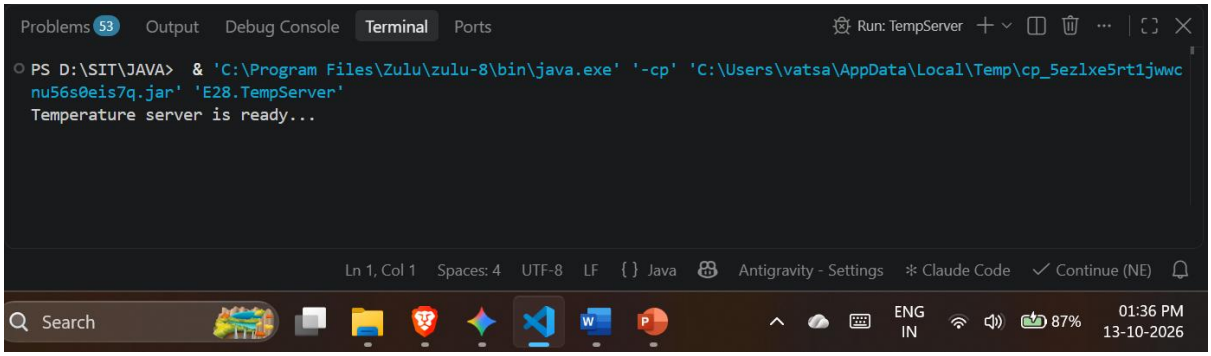
```
package E28;
import java.lang.*;
import java.rmi.*;
import java.util.Scanner;

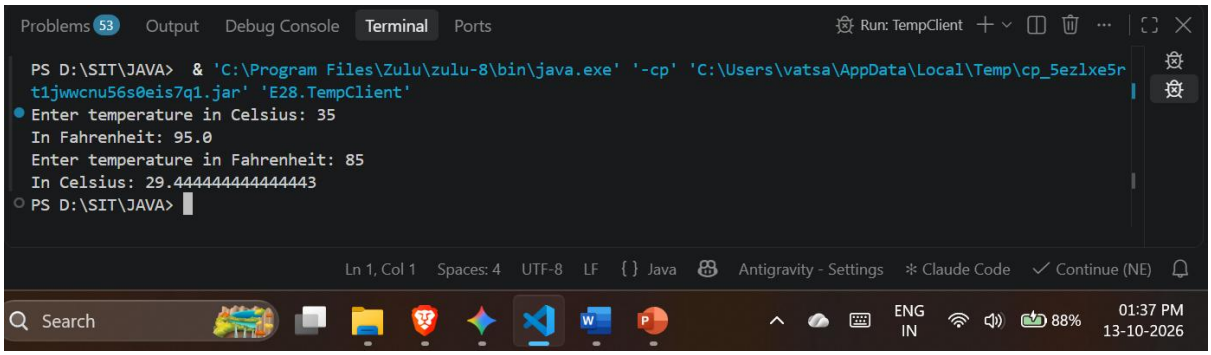
public class TempClient {
   public static void main(String[] args) throws Exception {
      Converter c = (Converter) Naming.lookup("rmi://localhost/TempService");
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter temperature in Celsius: ");
      double cel = sc.nextDouble();
      System.out.println("In Fahrenheit: " + c.celsiusToFahrenheit(cel));

      System.out.print("Enter temperature in Fahrenheit: ");
      double fah = sc.nextDouble();
      System.out.println("In Celsius: " + c.fahrenheitToCelsius(fah));
   }
}
```

Output:





Conclusion:

I learned how to develop distributed Java applications using Remote Method Invocation (RMI) by defining remote interfaces and their implementations. I learned how to utilize stubs and skeletons to allow a client program to invoke methods on a remote server as if they were local objects. I learned how to start and use the RMI registry to bind remote services so the client can locate and communicate with them over the network. Finally, I learned how to apply these concepts practically by building functional client-server applications for basic greetings, arithmetic calculations, and temperature conversions.